Dive into Deep Learning (D2L)

Chapter 3: Linear Neural Networks for Regression

Joon Soo Yoo, Ph.D.

Spring 2026

Assistant Professor
Defense & Advanced Security (DAS) Lab
Jeonbuk National University (JBNU)

3.1 Linear Regression

선형 회귀: 수치 예측의 기초

3.1 Linear Regression: 수치 예측의 세계

- **회귀(Regression)**: 연속적인 숫자를 예측하는 모든 문제
  - 예: 집값/주가 예측, 환자 체류 기간, 수요 예측 등
- **분류(Classification)**와 대조됨: 범주(Category)를 맞추는 문제

핵심 가정 (Linearity Assumption)

"입력 특징(x)과 정답(y) 사이의 관계는 가중치들의 합으로 표현될 만큼 직선적일 것이다."

3.1.1 Terminology: 데이터의 구성

데이터셋 용어 사전

용어	설명
Training Set	모델이 학습하는 데 사용되는 데이터 전체 집합
Example / Sample	데이터셋의 한 줄 (Row), 개별 데이터 포인트 (n개)
Feature (Covariate)	예측의 근거가 되는 변수 (예: 면적, 연식) (d개)
Label (Target)	모델이 맞추고자 하는 실제 정답 (예: 매매가)

3.1.1.1 Linear Model: 어파인 변환(Affine Transformation)

- **모델식**: $\text{price} = w_{\text{area}} \cdot \text{area} + w_{\text{age}} \cdot \text{age} + b$

파라미터(Parameters)의 역할

- **Weights (w)**: 각 특징이 예측값에 미치는 영향력(기울기)
- **Bias (b)**: 모든 입력이 0일 때의 기본값(절편). 원점을 지나지 않는 직선을 표현하기 위해 필수적

수학적 성질

선형 변환(Linear Transformation)에 평행 이동(Translation)인 b를 더한 형태를 **어파인 변환(Affine Transformation)**이라고 함

3.1.1.1 행렬 연산으로의 확장 (Vectorization)

- 특징이 수백 개로 늘어나면 개별 수식은 비효율적 → 벡터/행렬 표현 도입

벡터 표현 (샘플 1개)

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

- $\mathbf{x} \in \mathbb{R}^d$: 입력 벡터
- $\mathbf{w} \in \mathbb{R}^d$: 가중치 벡터

행렬 표현 (전체 데이터)

$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} + b$$

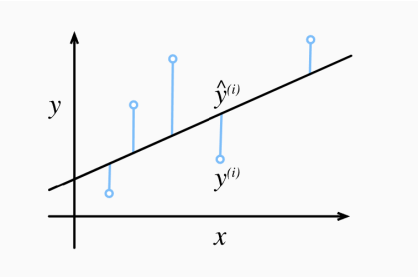
- $\mathbf{X} \in \mathbb{R}^{n \times d}$: Design Matrix
- b는 브로드캐스팅 적용

3.1.1.2 Visualization: 데이터에 선 긋기

- **목표**: 흩어진 데이터 포인트들을 가장 잘 설명하는 하나의 직선을 찾는 것

그래프의 구성 요소

- **Blue Points**: 실제 관측된 데이터 $(x^{(i)}, y^{(i)})$
- **Red Line**: 모델이 예측한 직선 $\hat{y} = wx + b$
- **Vertical Lines**: 오차(Residuals). 예측값과 실제값의 차이



[Figure 3.1.1: Linear Fit]

핵심 직관

직선과 점 사이의 **수직 선분들의 길이의 합(정확히는 제곱합)**을 최소로 만드는 과정이 곧 모델의 학습임

3.1.1.2 Loss Function: "얼마나 못했니?"

- **개념**: 모델의 예측값($\hat{y}$)과 실제 정답(y) 사이의 거리를 측정
- **특징**: 항상 0 이상의 값을 가지며, 완벽한 예측 시 0이 됨

제곱 오차 (Squared Error)

i번째 데이터 샘플에 대한 오차 $l^{(i)}$:

$$l^{(i)}(\mathbf{w}, b) = \frac{1}{2}\left(\hat{y}^{(i)} - y^{(i)}\right)^2$$

- **상수 1/2의 비밀**: 나중에 이 식을 미분할 때, 지수 2와 곱해져 계산을 깔끔하게(1로) 만들기 위한 수학적 배려임

3.1.1.2 제곱 오차의 특성: "양날의 검"

- **장점**: 오차가 커질수록 손실이 기하급수적으로 커짐 → 모델이 **"큰 실수"를 하지 않도록 강력하게 압박**함
- **단점**: 이상치(Outlier)에 지나치게 민감함 → 엉터리 데이터 하나 때문에 전체 모델이 흔들릴 수 있음

기하학적 해석

데이터 포인트와 회귀 직선 사이의 수직 거리(Residual)를 제곱하여 면적으로 합산하는 것과 같음

3.1.1.2 전체 손실과 최적화 (Optimization)

- 개별 오차를 모두 더해 데이터 개수(n)로 나누어 평균을 냄

평균 손실 (Total Loss)

$$L(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} l^{(i)}(\mathbf{w}, b) = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{2}\left(\mathbf{w}^\top \mathbf{x}^{(i)} + b - y^{(i)}\right)^2$$

우리의 최종 목적지

이 전체 손실 L을 최소화하는 환상의 콤비 $(\mathbf{w}^*, b^*)$를 찾는 것:

$$\mathbf{w}^*, b^* = \underset{\mathbf{w}, b}{\operatorname{argmin}}\, L(\mathbf{w}, b)$$

3.1.1.4 최적화(Optimization): 반복의 미학

- **Gradient Descent (경사 하강법)**: 손실 함수를 낮추는 방향으로 파라미터를 점진적으로 업데이트하여 오차를 줄여가는 과정
- **반복적 접근 (Iterative Approach)**: 한 번에 정답을 찾는 대신, 여러 번의 작은 수정을 거쳐 최적해에 도달함

핵심 아이디어

"현재 위치에서 가장 가파른 내리막길(Negative Gradient)로 한 걸음 내디뎌라."

3.1.1.4 왜 '미니배치'인가? (전략 비교)

GD vs SGD vs Minibatch SGD

방식	데이터 사용량	장단점
Batch GD	전체 데이터셋	안정적이지만 대규모 데이터에서 매우 느림
Stochastic GD	샘플 1개	빠르지만 하드웨어 효율 저하 및 경로 불안정
Minibatch SGD	**일부 데이터 (묶음)**	**GPU 병렬 연산에 최적화된 표준 방식**

- **컴퓨터 구조적 이유**: CPU/GPU는 단일 연산보다 **행렬 연산(Vectorization)**을 훨씬 빠르게 처리함. 따라서 샘플을 묶어서 처리하는 것이 압도적으로 유리함.

3.1.1.4 알고리즘: 어떻게 업데이트하는가?

1. 파라미터를 무작위로 초기화함
2. 무작위 미니배치 $\mathcal{B}_t$를 샘플링함
3. 해당 배치의 평균 기울기를 계산하여 업데이트 수행

업데이트 공식 (Update Rule)

$$(\mathbf{w}, b) \leftarrow (\mathbf{w}, b) - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \partial_{(\mathbf{w}, b)} l^{(i)}(\mathbf{w}, b)$$

- η (Learning Rate): 보폭의 크기
- $|\mathcal{B}|$ (Batch Size): 묶음의 크기 (보통 32 256 사이의 2의 거듭제곱 사용)

3.1.1.4 그레이디언트 벡터(∇L)의 해부

- **개념**: 모든 파라미터($w_1, w_2, \ldots, b$)에 대한 개별 편미분 값을 하나로 묶은 것

그레이디언트 벡터의 구성

$$\nabla_{(\mathbf{w},b)} L(\mathbf{w}, b) = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b} \end{bmatrix}$$

각 성분의 의미

- $\frac{\partial L}{\partial w_1}$: 다른 값은 고정하고 w_1만 살짝 바꿨을 때, 전체 손실(L)이 변하는 정도
- 이 벡터가 가리키는 방향이 곧 **"손실이 가장 빠르게 증가하는 방향"**임

3.1.1.5 Predictions: 학습된 모델 활용하기

- **개념**: 최적화된 파라미터($\hat{\mathbf{w}}, \hat{b}$)를 사용하여 한 번도 보지 못한 데이터의 정답을 추정
- **예시**: 면적(x_1)과 연식(x_2)을 넣어서 새로운 집값($\hat{y}$) 계산

수학적 표현

$$\hat{y} = \hat{\mathbf{w}}^\top \mathbf{x} + \hat{b}$$

3.1.2 Vectorization: 속도를 위한 벡터화

- **목표**: 미니배치의 수많은 샘플을 동시에(simultaneously) 처리
- **문제**: 파이썬의 `for-loop`는 연산마다 오버헤드가 발생하여 매우 느림
- **해결**: 최적화된 선형대수 라이브러리(C++/CUDA 백엔드)를 사용하는 벡터화 연산 도입

실험 준비: 10000차원 벡터 생성

```
import torch
import time

n = 10000
a = torch.ones(n)
b = torch.ones(n)
```

- 1로 채워진 두 개의 거대 벡터를 서로 다른 두 방식으로 더해보고 시간을 비교함

3.1.2 벤치마킹: 루프 vs 벡터 연산

방법 1: Python For-loop (비효율적)

```
c = torch.zeros(n)
t = time.time()
for i in range(n):
    c[i] = a[i] + b[i]
print(f'{time.time() - t:.5f} sec')

# 결과: 0.17802 sec
```

방법 2: Vectorized Addition (효율적)

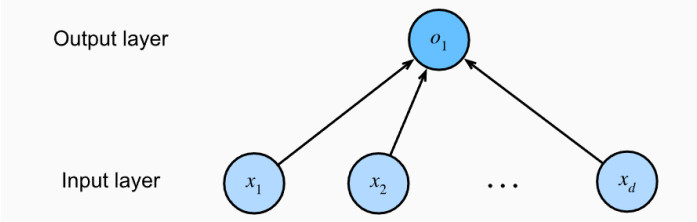
```
t = time.time()
d = a + b
print(f'{time.time() - t:.5f} sec')

# 결과: 0.00036 sec
```

성능 차이 분석

벡터화된 연산(0.00036s)이 루프(0.17802s)보다 **약 494배** 빠름! 연산의 복잡도가 커질수록 이 격차는 더욱 벌어짐.

3.1.1.6 Architecture: 신경망으로 본 선형 회귀

- **구조적 특징**: d개의 입력 뉴런이 하나의 출력 뉴런에 완전히 연결됨
- **연산**: 각 연결선은 가중치(w_i)를 가지며, 뉴런에서 합산($\sum$)이 일어남



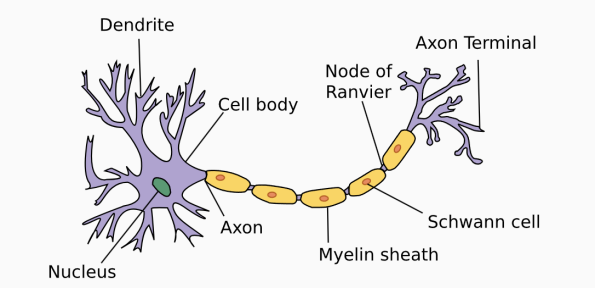
[Figure 3.1.2: Linear regression is a single-layer fully connected neural network]

Terminology in Networks

- **Input Layer**: $x_1, x_2, \ldots, x_d$ (입력 차원 / Feature dimensionality)
- **Fully Connected**: 모든 입력이 출력에 연결된 조밀한 구조

3.1.4.1 Biology: 인공 뉴런의 모태

- **역사**: 1943년 McCulloch와 Pitts가 생물학적 뉴런의 수학적 모델을 처음 제안함
- **구조**: 수상돌기(Dendrites), 세포체(Nucleus), 축삭(Axon)으로 구성



[Figure 3.1.3: The anatomy of a real biological neuron]

생물학적 신호 전달

다른 뉴런으로부터 온 신호(x_i)가 시냅스 가중치(w_i)에 의해 증폭되거나 억제된 후, 세포체에서 합산되어 전달됨

3.1.4.1 Mapping: 생물학적 구조와 수학적 모델

- 선형 회귀의 수식 $y = \sum w_i x_i + b$는 뉴런의 물리적 활동을 추상화한 것임

Biological vs Artificial Analogy

Biological (생물)	Artificial (인공)
Dendrites (수상돌기)	입력값 $(x_1, x_2, \ldots, x_d)$ 수신
Synapses (시냅스)	가중치 $(w_1, w_2, \ldots, w_d)$ - 신호 세기 조절
Nucleus (세포체)	가중 합산 연산 $(\sum w_i x_i + b)$
Axon (축삭)	출력값 전달 및 활성화 $(\sigma(y))$

3.2 Object-Oriented Design for Implementation

3.2.0.1 Class vs. Object: 설계도와 실체

- **Class**: 객체를 만들기 위한 **설계도 (Blueprint)**
- **Object (Instance)**: 설계도에 따라 메모리에 구현된 **실체**

비유로 이해하기

- **Class**: '드론 설계도' (프로펠러 개수, 센서 사양 등 정의)
- **Object**: '내 책상 위의 DAS-01', '옆 실험실의 DAS-02'

Python Syntax

`class Drone:` → 클래스 정의
`my_drone = Drone()` → 객체 생성 (인스턴스화)

3.2.0.2 Attributes: 객체의 속성

- **Attribute (속성)**: 객체가 내부에 저장하고 있는 **데이터/변수**
- `__init__` 메서드를 통해 객체 생성 시 초기화함

Python Code: 속성 정의

```
class Drone:
    def __init__(self, model, battery):
        self.model = model      # 속성 1: 모델명
        self.battery = battery  # 속성 2: 배터리 잔량

# 객체 생성 시 데이터 전달
drone_a = Drone("DAS-Alpha", 100)
print(drone_a.model) # 출력: DAS-Alpha
```

3.2.0.3 Methods: 객체의 기능

- **Method**: 클래스 내부에서 정의된 **함수**
- 객체의 속성을 활용하여 특정 동작을 수행함

Python Code: 메서드 정의

```python
class Drone:
    # ... (생략) ...
    def fly(self):
        if self.battery > 0:
            print(f"{self.model}가 이륙합니다!")
            self.battery -= 10
        else:
            print("배터리가 부족합니다.")

drone_a.fly() # 메서드 호출
```

3.2.0.4 요약: Class의 구조

Components of a Python Class

용어	설명	비유
Class	객체를 정의하는 틀	붕어빵 틀
Object	클래스로부터 생성된 실체	붕어빵
Attribute	객체의 상태를 나타내는 변수	붕어빵의 내용물 (팥, 슈크림)
Method	객체의 동작을 나타내는 함수	붕어빵의 맛 (달콤함)
Constructor	`__init__` (초기화 메서드)	재료 넣기

중요 포인트

클래스 내부의 모든 메서드는 첫 번째 인자로 반드시 `self`를 받아야 함!

[Exercise] 클래스 설계: 전사(Warrior) 키우기

- **상황**: 간단한 RPG 게임의 전사 클래스를 설계합니다.

문제 요구사항

1. `Warrior` 클래스를 생성하세요.
2. `__init__`에서 이름(`name`), 체력(`hp`), 공격력(`power`)을 초기화하세요.
3. `attack(self, target)` 메서드를 만드세요.
   - `target`은 공격받는 다른 `Warrior` 객체입니다.
   - 공격 시 `target.hp`에서 `self.power`만큼 차감합니다.
   - 공격 상황과 대상의 남은 체력을 출력하세요.

[Solution] Warrior 클래스 구현 정답

Python Code: 객체 간 상호작용

```python
class Warrior:
    def __init__(self, name, hp, power):
        self.name = name
        self.hp = hp
        self.power = power

    def attack(self, target):
        target.hp -= self.power # 상대방의 체력 감소
        print(f" [{self.name}]이(가) [{target.name}]을 공격!")

        if target.hp <= 0:
            print(f" [{target.name}]이(가) 쓰러졌습니다.")
        else:
            print(f"[{target.name}] 남은 체력: {target.hp}")

# 인스턴스 생성 및 테스트
p1 = Warrior("아라곤", 100, 25)
p2 = Warrior("오크", 40, 10)
p1.attack(p2)
```

Summary

- **객체 전달**: target 인자로 문자열이 아닌 **객체 자체**가 넘어옵니다.
- **상태 변화**: 한 객체의 메서드가 다른 객체의 속성(hp)을 직접 수정합니다.
- **독립성**: p1과 p2는 같은 클래스 출신이지만 데이터는 각자 관리합니다.

3.2 Object-Oriented Design: 효율적인 구현을 위한 설계

- **배경**: 선형 회귀는 단순하지만, 딥러닝 훈련에 필요한 모든 구성 요소(데이터, 모델, 손실함수, 최적화)를 이미 포함하고 있음
- **목표**: 이 구성 요소들을 독립적인 **객체(Object)**로 정의하여 복잡한 모델로 확장 가능하게 함
- **영감**: **PyTorch Lightning**과 같은 최신 오픈소스 라이브러리의 설계 철학을 반영

객체 지향의 장점

코드의 가독성 향상, 컴포넌트 간 상호작용의 명확화, 하드웨어(CPU/GPU) 환경에 구애받지 않는 유연한 훈련 가능

3.2 핵심 아키텍처: 세 가지 기둥

The 3 Core Classes in D2L

클래스	역할 및 포함 내용
Module	모델 구조, 손실 함수(Loss), 최적화 방법(Optimization) 정의
DataModule	학습(Training) 및 검증(Validation)을 위한 데이터 로더 공급
Trainer	Module과 DataModule을 결합하여 실제 학습 루프를 실행 (하드웨어 제어)

3.2 API 디자인: 훈련의 자동화

- **Module & DataModule**: 모델 개발자가 가장 자주 수정하고 다루게 되는 핵심 영역
- **Trainer**: 병렬 처리, 하드웨어 가속(GPU), 최적화 알고리즘의 세부 구현을 담당

Interaction Flow

1. `DataModule`이 데이터를 준비함
2. `Module`이 모델의 예측과 오차를 계산함
3. `Trainer`가 두 객체를 연결하여 반복적으로 파라미터를 업데이트함

구현의 일관성

"이후의 모든 모델(MLP, CNN, RNN 등)은 동일한 API 구조를 따르게 되므로, 한 번 익혀두면 확장이 매우 쉬워짐"

3.2.1 Utilities: 왜 이런 도구가 필요한가?

- **문제점**: 주피터 노트북에서 클래스 정의가 너무 길어지면 가독성이 떨어짐
- **요구사항**: 짧은 코드 조각 + 풍부한 설명이 섞인 형태가 이상적임
- **해결책**: 클래스를 먼저 선언하고, 메서드를 나중에 "등록"하는 유틸리티 도입

핵심 기능

클래스 정의가 끝난 후, 심지어 **인스턴스가 이미 생성된 후에도** 새로운 기능을 클래스에 붙일 수 있음 (Dynamic Patching)

3.2.1 add_to_class의 구현

- 파이썬의 내장 함수 setattr을 사용하여 동적으로 속성을 부여함

Python Code: 유틸리티 정의

```python
def add_to_class(Class):  #@save
    """함수를 생성된 클래스의 메서드로 등록하는 데코레이터"""
    def wrapper(obj):
        # setattr(대상, 이름, 값)
        setattr(Class, obj.__name__, obj)
    return wrapper
```

- obj.__name__: 함수의 원래 이름 (예: 'do')을 문자열로 가져옴
- obj: 함수 객체 자체를 값으로 할당

3.2.1 활용 예시: 클래스 확장하기

Step 1: 기본 클래스와 인스턴스 생성

```
class A:
    def __init__(self):
        self.b = 1

a = A() # 인스턴스 미리 생성
```

Step 2: 외부에서 메서드 주입

```
@add_to_class(A)
def do(self):
    print('Class attribute "b" is', self.b)

a.do() # 출력: Class attribute "b" is 1
```

포인트

데코레이터를 사용해 `A` 클래스에 `do()`를 보냈으므로, 이미 생성된 `a`도 이 기능을 즉시 사용할 수 있음

[Exercise] 클래스 기능의 동적 확장

- **상황**: 드론 제어 시스템의 기초 클래스를 설계한 후, 나중에 '보안 이륙' 기능을 추가해야 합니다.

문제 요구사항

1. `add_to_class` 유틸리티 함수를 정의하세요.
2. `DefenseDrone` 클래스를 만드세요. (`__init__`에서 `self.drone_id`를 저장합니다.)
3. `@add_to_class`를 사용하여 `secure_takeoff(self)` 메서드를 클래스 외부에 정의하고 추가하세요.
4. 메서드는 `"[ID] Drone: 보안 이륙을 시작합니다."`를 출력해야 합니다.

코드 뼈대

```
# 1. add_to_class 정의
# (여기에 코드를 작성하세요)

# 2. 클래스 정의 및 인스턴스 생성
# (여기에 코드를 작성하세요)

# 3. @add_to_class를 활용한 메서드 추가
# (여기에 코드를 작성하세요)
```

[Solution] 드론 클래스 확장 예시

Python Code: 문제 풀이

```python
def add_to_class(Class):
    def wrapper(obj):
        setattr(Class, obj.__name__, obj)
    return wrapper

class DefenseDrone:
    def __init__(self, drone_id):
        self.drone_id = drone_id

my_drone = DefenseDrone("DAS-01")

@add_to_class(DefenseDrone)
def secure_takeoff(self):
    print(f"[{self.drone_id}] Drone: 보안 이륙을 시작합니다.")

my_drone.secure_takeoff()
# 출력: [DAS-01] Drone: 보안 이륙을 시작합니다.
```

핵심 포인트 확인

- **self의 연결**: 클래스 밖에서 정의되었지만, `self`를 통해 `DefenseDrone`의 멤버 변수인 `drone_id`에 완벽히 접근함
- **실시간 반영**: 이미 생성된 `my_drone` 객체에 새로운 기능이 즉시 '주입'됨

3.2.1.(1) Inheritance: 부모의 능력을 물려받기

- **정의**: 기존 클래스(**Base/Parent**)의 속성과 메서드를 새로운 클래스(**Derived/Child**)가 그대로 물려받는 것
- **목적**: 코드 중복 제거 및 계층적 구조 설계

비유: 드론 시스템 개발

- **기본 드론 (Parent)**: 비행 기능, 배터리 관리 (공통 기능)
- **보안 드론 (Child)**: 기본 드론 기능 + **암호화 통신**
- **공격 드론 (Child)**: 기본 드론 기능 + **타겟 추적**

3.2.1.(2) 상속 문법: 괄호 하나로 끝나는 상속

- 클래스를 정의할 때 이름 뒤 괄호 () 안에 부모 클래스명을 넣음

Python Code: 기본 상속 예시

```python
class Drone: # 부모 클래스
    def fly(self):
        print("하늘을 납니다.")

class DefenseDrone(Drone): # Drone을 상속받음
    def patrol(self):
        print("구역을 정찰합니다.")

my_drone = DefenseDrone()
my_drone.fly()    # 부모에게 물려받은 기능 (따로 안 짜도 됨!)
my_drone.patrol() # 자신만의 고유 기능
```

3.2.1.(3) Overriding & `super()`: 기존 기능 고쳐쓰기

- **Overriding**: 부모의 메서드를 자식 입맛에 맞게 재정의함
- `super()`: 부모 클래스의 메서드를 호출할 때 사용 (확장의 핵심)

Python Code: 기능을 덮어쓰고 덧붙이기

```
class DefenseDrone(Drone):
    def fly(self):
        # 1. 부모의 원래 비행 기능을 먼저 실행
        super().fly()
        # 2. 자식만의 보안 기능을 덧붙임
        print("보안 모드로 비행을 시작합니다.")

d = DefenseDrone()
d.fly()
# 출력 결과:
# 하늘을 납니다.
# 보안 모드로 비행을 시작합니다.
```

핵심 포인트

`super()`를 쓰지 않으면 부모의 기능은 완전히 무시(Replace)되지만, `super()`를 쓰면 부모의 기능을 기반으로 기능을 확장(Extend)할 수 있음!

3.2.1.(4) 왜 딥러닝에서 상속이 중요한가?

- 우리가 앞으로 구현할 모든 신경망은 `nn.Module`을 상속받아야 함

D2L & PyTorch 상속 체계

상속받는 대상	이유
nn.Module	파라미터 관리, GPU 이동, 오차 역전파 기능을 쓰기 위해
HyperParameters	하이퍼파라미터 자동 저장 기능을 쓰기 위해
DataModule	표준화된 데이터 로딩 형식을 맞추기 위해

팁

상속은 거인의 어깨 위에 올라타는 것과 같음. 파이토치가 이미 만들어놓은 강력한 엔진(`nn.Module`)을 물려받고, 이 위에서 모델 설계라는 본질에만 집중할 수 있음.

3.2.1.(5) [Exercise] 전사의 진화: Berserker 클래스

- **상황**: 기본 전사(`Warrior`) 클래스를 상속받아, 더 강력한 기술을 쓰는 광전사(`Berserker`)를 만들려고 합니다.

문제 요구사항

1. `Warrior` 클래스를 상속받는 `Berserker` 클래스를 정의하세요.
2. `attack()` 메서드를 오버라이딩(Overriding) 하세요.
3. `super()`를 사용하여 부모의 `attack()` 기능을 먼저 실행한 뒤, 광폭화: 추가 데미지를 입힙니다!"라는 문구를 출력하게 만드세요.

코드 뼈대

```
class Warrior:
    def attack(self):
        print("기본 공격을 수행합니다.")


# 여기에 Berserker 클래스를 구현하세요.
```

3.2.1.(6) [Solution] super()를 활용한 기능 확장

Python Code: 광전사 구현

```python
class Warrior:
    def attack(self):
        print("기본 공격을 수행합니다.")

class Berserker(Warrior):
    def attack(self):
        # 1. 부모의 기본 공격 기능을 그대로 가져옴
        super().attack()
        # 2. 광전사만의 특수 효과를 덧붙임
        print("광폭화: 추가 데미지를 입힙니다!")

# 테스트
hero = Berserker()
hero.attack()

# 출력 결과:
# 기본 공격을 수행합니다.
# 광폭화: 추가 데미지를 입힙니다!
```

팁

- **Extension (확장)**: `super().attack()`이 없었다면 기본 공격 멘트는 사라졌을 것입니다.
- **효율성**: 부모의 로직이 아무리 복잡해도 `super()` 한 줄이면 재사용이 가능합니다.
- **일관성**: 부모 클래스의 `attack` 내용이 나중에 바뀌더라도, `super()`를 쓴 자식 클래스들은 자동으로 업데이트된 내용을 반영하게 됩니다.

3.2.1.2 HyperParameters: 하이퍼파라미터 자동 저장

- **문제**: 모델이 복잡해질수록 `__init__`에서 변수를 할당하는 반복 코드가 늘어남
- **해결**: 베이스 클래스를 상속받아 단 한 줄의 코드로 모든 인자를 속성으로 등록

전형적인 불편한 방식 (Boilerplate)

```
class MyModel:
    def __init__(self, a, b, c, d, e, f):
        self.a = a
        self.b = b
        self.c = c
        # ... (변수가 많아지면 코드가 매우 길어짐)
```

D2L의 제안

`HyperParameters`를 상속받고 `save_hyperparameters()`를 호출하면 끝!

3.2.1.2 활용 예시: 상속과 자동 할당

Python Code: 클래스 B의 정의 및 테스트

```python
# d2l.HyperParameters를 상속받음
class B(d2l.HyperParameters):
    def __init__(self, a, b, c):
        # c는 속성으로 저장하지 않고 무시함
        self.save_hyperparameters(ignore=['c'])

        print('self.a =', self.a, 'self.b =', self.b)
        print('Is self.c defined?', hasattr(self, 'c'))

# 객체 생성
b = B(a=1, b=2, c=3)
# 출력:
# self.a = 1 self.b = 2
# Is self.c defined? False
```

기능 요약

- **자동화**: 별도의 대입 연산 없이 `self.a`, `self.b`가 생성됨
- **필터링**: `ignore` 인자를 통해 특정 변수만 제외 가능

3.2.1.2 HyperParameters: 요약 및 결론

- **상속의 힘**: `d2l.HyperParameters`로부터 "자동 저장 기술"을 전수받음
- **자동화**: `self.a = a`와 같은 단순 반복 코드를 완전히 제거
- **유연성**: `ignore` 인자를 통해 특정 변수만 선택적으로 저장 가능

Why it matters?

모델의 구조가 복잡해지고 파라미터가 수십 개로 늘어나도, **코드의 길이는 늘어나지 않으며** 가독성이 최상으로 유지됨.

3.2.1.2 유틸리티의 의의: 확장성(Scalability)

- **Implicit Extension**: 나중에 하이퍼파라미터가 추가되어도 `save_hyperparameters()`가 알아서 처리하므로 코드를 수정할 필요가 없음
- **유지보수**: 모델의 인자가 많아져도 `__init__` 함수 본문이 매우 깔끔하게 유지됨

Why use save_hyperparameters?

장점	내용
Clean Code	수십 개의 변수 할당 코드가 단 한 줄로 축소됨
Flexibility	훈련 중에 특정 파라미터만 골라 수정하기 용이함
Readability	모델의 핵심 로직에 더 집중할 수 있게 해줌

3.2.2 d2l 패키지 설치 오류 해결

- **문제 상황**: 표준 `!pip install d2l` 실행 시, 구버전 numpy (예: 1.23.x) 설치 시도 중 **'subprocess-exited-with-error'** 발생
- **원인**: 최신 파이썬 환경(3.11 이상)과 d2l이 요구하는 구형 의존성(Dependencies) 간의 빌드 충돌

Python Code: 단계별 해결 방법

```python
# 1단계: 빌드 패키지 최신화 (안전한 설치를 위해 권장)
!pip install --upgrade pip setuptools wheel

# 2단계: 호환 가능한 최신 numpy 미리 설치 (충돌 원인 제거)
!pip install numpy>=1.24.0

# 3단계: d2l만 단독 설치 (의존성 검사 생략 옵션 활용)
!pip install d2l --no-deps
```

반드시 수행 필수!

설치 완료 후, 상단 메뉴의 **[런타임]** → **[세션 다시 시작]** (또는 Restart Kernel)을 눌러 파이썬이 변경 사항을 인식하게 해야 함

3.2.2 ProgressBoard: 실시간 모니터링의 필요성

- **배경**: 수천 번의 반복(Iteration)이 일어나는 딥러닝 훈련에서 숫자로 된 로그만으로는 학습 상태 파악이 어려움
- **역할**: 훈련 손실(Training Loss)과 검증 손실(Validation Loss)을 실시간으로 비교 시각화
- **구조**: `d2l.HyperParameters`를 상속받아 설정값(축 범위, 눈금 등)을 관리함

주요 초기화 파라미터

파라미터	설명
xlabel, ylabel	각 축의 이름 (예: 'epoch', 'loss')
xlim, ylim	그래프의 표시 범위 설정
xscale, yscale	'linear' 또는 'log' 스케일 선택 가능

3.2.1.2 핵심 기능: `draw`와 `every_n`

- `draw(x, y, label, every_n)`: 특정 좌표에 점을 찍고 선으로 연결함

Smoothing (평활화) 원리

매 배치(Batch)마다 Loss가 요동치는 경우 그래프가 매우 지저분해짐 (Noisy). `every_n`은 인접한 n개의 데이터를 평균 내어 하나의 점으로 표현함.

- **효과**: 전체적인 학습의 **경향성(Trend)**을 훨씬 명확하게 파악할 수 있음

3.2.2 실전 예제: 애니메이션 테스트

Python Code: sin과 cos 곡선 그리기

```
board = d2l.ProgressBoard('x')
for x in np.arange(0, 10, 0.1):
    # 'sin' 곡선: 2개마다 평균 (약간의 노이즈 허용)
    board.draw(x, np.sin(x), 'sin', every_n=2)

    # 'cos' 곡선: 10개마다 평균 (매우 부드럽게 표현)
    board.draw(x, np.cos(x), 'cos', every_n=10)
```

주의 사항

본 절의 `ProgressBoard.draw`는 뼈대만 정의되어 있으며, 실제 시각화 구현체는 부록(Section 23.7)에 포함되어 있음. 실제 실행 시에는 `d2l.ProgressBoard`를 사용하여 결과 확인 가능.

3.2.2 Module (1): 구조와 초기화 (Setup)

- **다중 상속**: nn.Module의 신경망 관리 능력과 HyperParameters의 저장 능력을 결합

Implementation: Part 1

```
class Module(nn.Module, d2l.HyperParameters):  #@save
    """The base class of models."""
    def __init__(self, plot_train_per_epoch=2, plot_valid_per_epoch=1):
        super().__init__()           # nn.Module 초기화 (필수!)
        self.save_hyperparameters()  # 인자 자동 저장
        self.board = ProgressBoard() # 실시간 시각화 전광판 생성

    def loss(self, y_hat, y):
        """손실 함수: 자식 클래스에서 구현 필요"""
        raise NotImplementedError

    def forward(self, X):
        """순전파 연산: 내부 네트워크(self.net)에 위임"""
        assert hasattr(self, 'net'), 'Neural network is not defined'
        return self.net(X)
```

3.2.2 Module (2): 학습 로직과 시각화 (Action)

Implementation: Part 2

```
def plot(self, key, value, train):
    """학습 진행 상황을 ProgressBoard에 플롯"""
    assert hasattr(self, 'trainer'), 'Trainer is not inited'
    self.board.xlabel = 'epoch'
    if train: # 훈련 단계의 X축 좌표 계산
        x = self.trainer.train_batch_idx / self.trainer.num_train_batches
        n = self.trainer.num_train_batches / self.plot_train_per_epoch
    else:     # 검증 단계의 X축 좌표 계산
        x = self.trainer.epoch + 1
        n = self.trainer.num_val_batches / self.plot_valid_per_epoch
    self.board.draw(x, value.to(d2l.cpu()).detach().numpy(),
                    ('train_' if train else 'val_') + key, every_n=int(n))

def training_step(self, batch):
    """한 번의 훈련 스텝: Loss 계산 및 플롯"""
    l = self.loss(self(*batch[:-1]), batch[-1]) # forward + loss
    self.plot('loss', l, train=True)
    return l

def validation_step(self, batch):
    """한 번의 검증 스텝"""
    l = self.loss(self(*batch[:-1]), batch[-1])
    self.plot('loss', l, train=False)

def configure_optimizers(self):
    """최적화 알고리즘 설정: 자식 클래스에서 구현 필요"""
    raise NotImplementedError
```

3.2.2 Module (1): 모델의 뼈대와 필수 메서드

- **다중 상속**: `nn.Module`(PyTorch) + `HyperParameters`(D2L)
- **역할**: 신경망의 파라미터를 관리하고 학습 루틴을 규격화함

Module 클래스의 3대 필수 메서드

메서드	역할
`__init__`	모델 파라미터 저장 및 시각화 도구(board) 준비
`training_step`	데이터 배치를 받아 손실(Loss)을 계산하고 반환
`configure_optimizers`	파라미터를 업데이트할 최적화 알고리즘(예: SGD) 설정

설계의 약속

`loss()`와 `configure_optimizers()`는 여기서 구현하지 않고 `raise NotImplementedError`로 남겨둠 → **"나중에 구체적인 모델을 만들 때 반드시 구현해라!"**라는 의미

3.2.3 DataModule: 데이터 공급의 표준화

- **역할**: 데이터 다운로드, 전처리, 그리고 배치(Batch) 단위 공급 담당
- **구조**: `d2l.HyperParameters`를 상속받아 경로 및 설정값 관리

Implementation: DataModule

```
class DataModule(d2l.HyperParameters):  #@save
    """데이터를 관리하는 베이스 클래스"""
    def __init__(self, root='../data', num_workers=4):
        self.save_hyperparameters() # root 경로, 병렬 처리 수 저장

    def get_dataloader(self, train):
        """데이터 로더 생성: 자식 클래스에서 구현 필수"""
        raise NotImplementedError

    def train_dataloader(self):
        return self.get_dataloader(train=True)

    def val_dataloader(self):
        return self.get_dataloader(train=False)
```

3.2.3 데이터 공급의 핵심: DataLoader

- **Python Generator**: 데이터를 한꺼번에 메모리에 올리지 않고, 필요할 때마다 한 배치씩 "생성"해서 공급
- **연결 고리**:

Module과 DataModule의 상호작용

주체	방향	내용
DataModule	→	`train_dataloader()`가 데이터 배치 공급
Module	←	`training_step(batch)`에서 배치를 받아 학습

병렬 처리 (num_workers)

CPU 코어를 여러 개 사용하여 데이터를 미리 읽어오는 기능을 조절함.

3.2.4 Trainer (1): 학습 환경 구축

- **역할**: 모델(Module)과 데이터(DataModule)를 결합하여 실제 학습 집도
- **주요 속성**: max_epochs (전체 데이터를 몇 번 반복해서 볼 것인가)

Implementation: Setup Methods

```
class Trainer(d2l.HyperParameters):
    def __init__(self, max_epochs, num_gpus=0, gradient_clip_val=0):
        self.save_hyperparameters() # 설정값 저장
        assert num_gpus == 0, 'No GPU support yet'

    def prepare_data(self, data):
        """DataModule로부터 데이터 로더를 가져와 준비"""
        self.train_dataloader = data.train_dataloader()
        self.val_dataloader = data.val_dataloader()
        self.num_train_batches = len(self.train_dataloader)

    def prepare_model(self, model):
        """모델에 트레이너 자신을 연결하고 시각화 범위 설정"""
        model.trainer = self
        model.board.xlim = [0, self.max_epochs]
        self.model = model
```

3.2.4 Trainer (2): 학습 실행 로직 (fit)

- `fit(model, data)`: 학습의 메인 엔트리 포인트
- **흐름**: 데이터 준비 → 모델 준비 → 최적화 설정 → 에폭 반복

Implementation: Fit Logic

```
def fit(self, model, data):
    self.prepare_data(data)
    self.prepare_model(model)
    self.optim = model.configure_optimizers() # 최적화 도구 획득
    self.epoch = 0

    for self.epoch in range(self.max_epochs):
        self.fit_epoch() # 한 에폭 동안의 학습 실행 (구현은 나중에)

def fit_epoch(self):
    """한 에폭의 상세 학습 단계: 자식 클래스에서 구현"""
    raise NotImplementedError
```

핵심 연결 고리

prepare_model에서 model.trainer = self라고 등록하는 덕분에, 모델 내부에서 트레이너의 상태(현재 몇 번째 배치인지 등)를 참조하여 그래프를 그릴 수 있음!

질의 & 응답

Any Question?